操作系统原理实验报告

实验名称	授课老师	学生姓名	学生学号
中断	张青	王健阳	21307261

一、实验要求

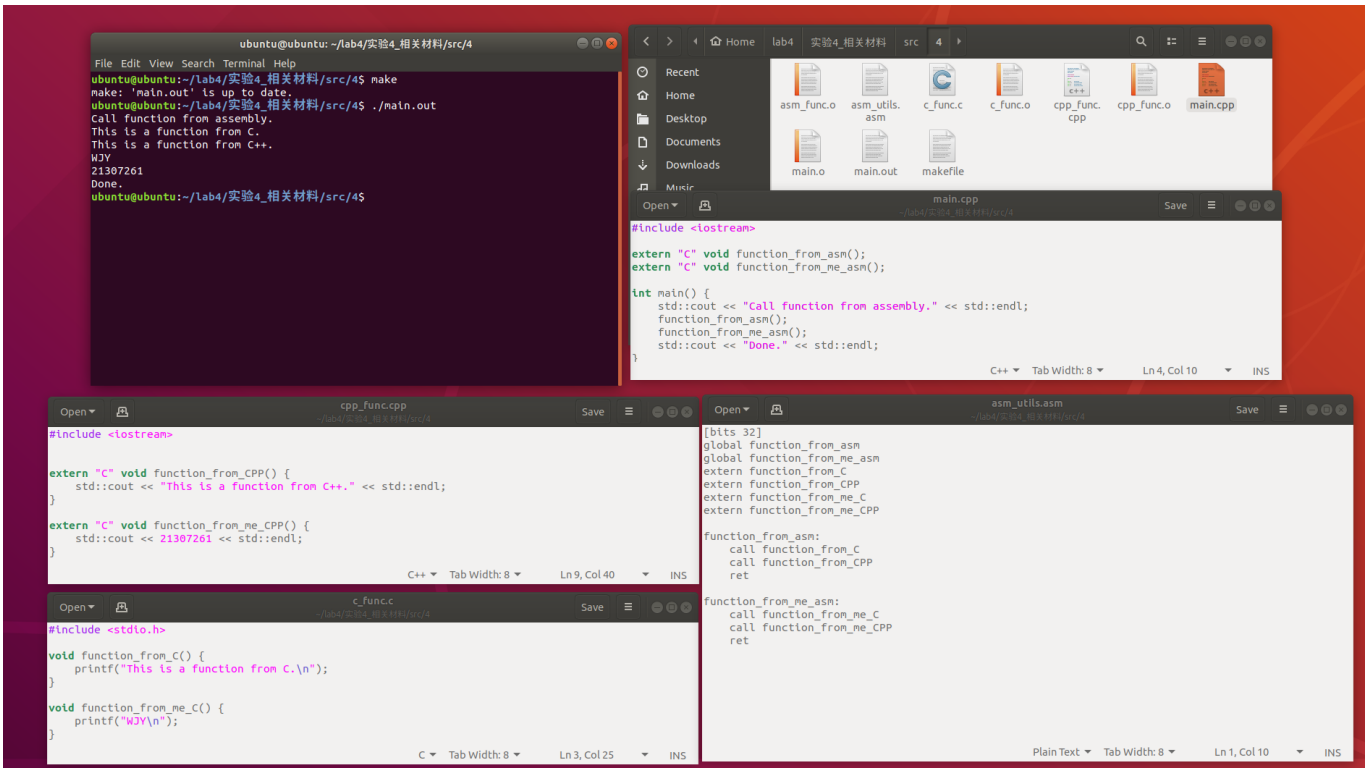
- 实验不限语言， C/C++/Rust都可以。
- 实验不限平台， Windows、Linux和MacOS等都可以。
- 实验不限CPU， ARM/Intel/Risc-V都可以。
- 基本了解混合编程的基本思路
- 基本了解中断的处理过程，以及用C实现时钟中断

二、实验过程

assignment 1

（为体验混合编程，适当添加了一些代码）

> 检查各个代码文件的内容
>
> 梳理内在组织逻辑
>
> 利用现成的makefile文件编译运行



（一些说明）

> `global`作用（C/C++调用汇编函数）

为了在链接阶段可以找到函数的实现，需要再汇编文件内将其声明为`global`

> `extern`作用（汇编调用C/C++函数）

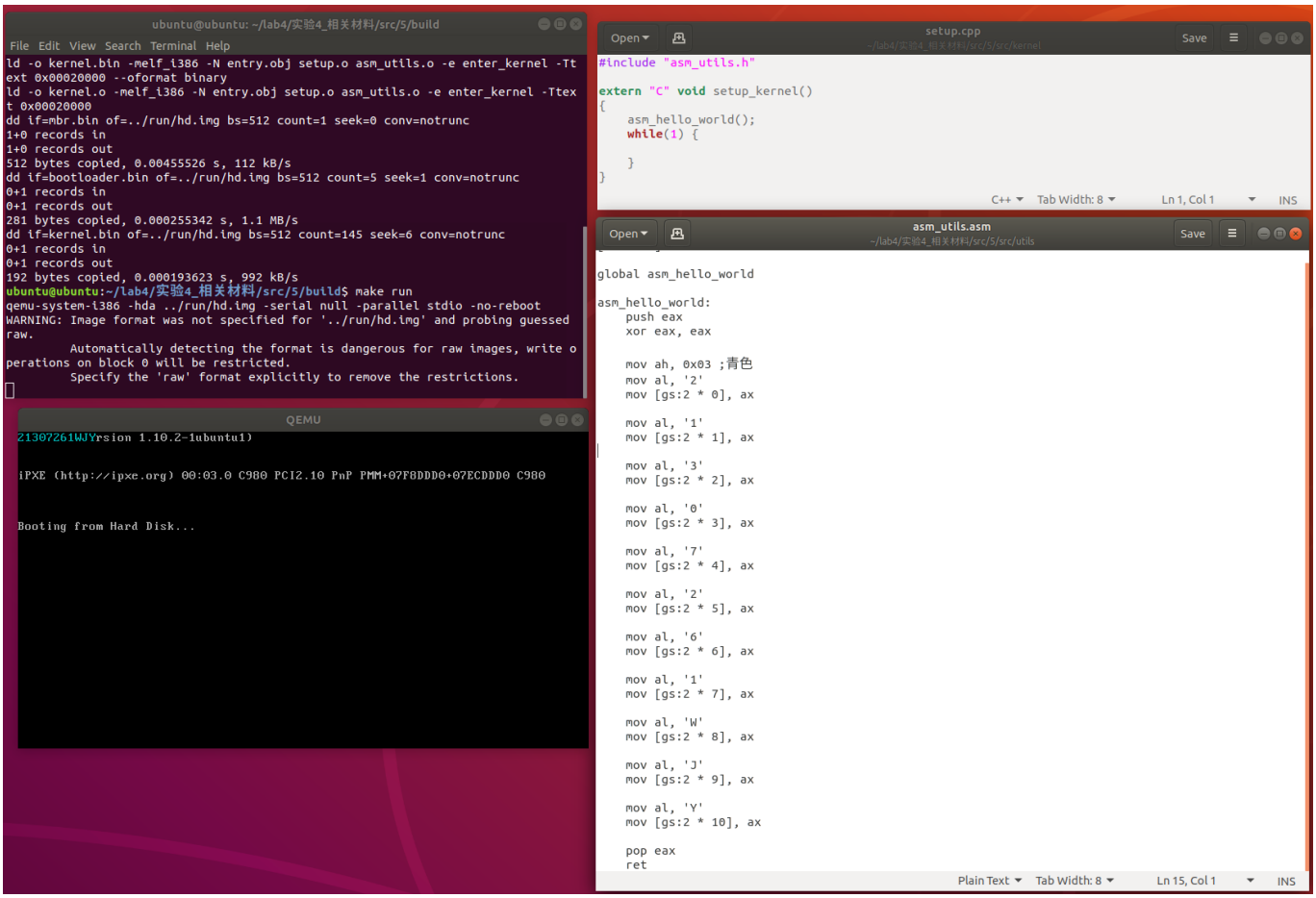
- 再汇编代码中使用C函数的时候，需要在汇编代码中声明此函数来自于外部
- 而如果是C++函数，则还需要在函数的定义和声明处加上`extern "C"`，因为C++支持重载，编译时会将函数名带上额外的信息。而`extern "C"`目的是告诉编译器按C代码的规则编译，不进行名字修饰。
- （C/C++调用汇编函数的时候，C/C++文件的函数定义也要加上`extern`）

> makefile

将编译时需要用到的语句依次打包写进makefile中，可以大大提高编译时的效率

assignment 2

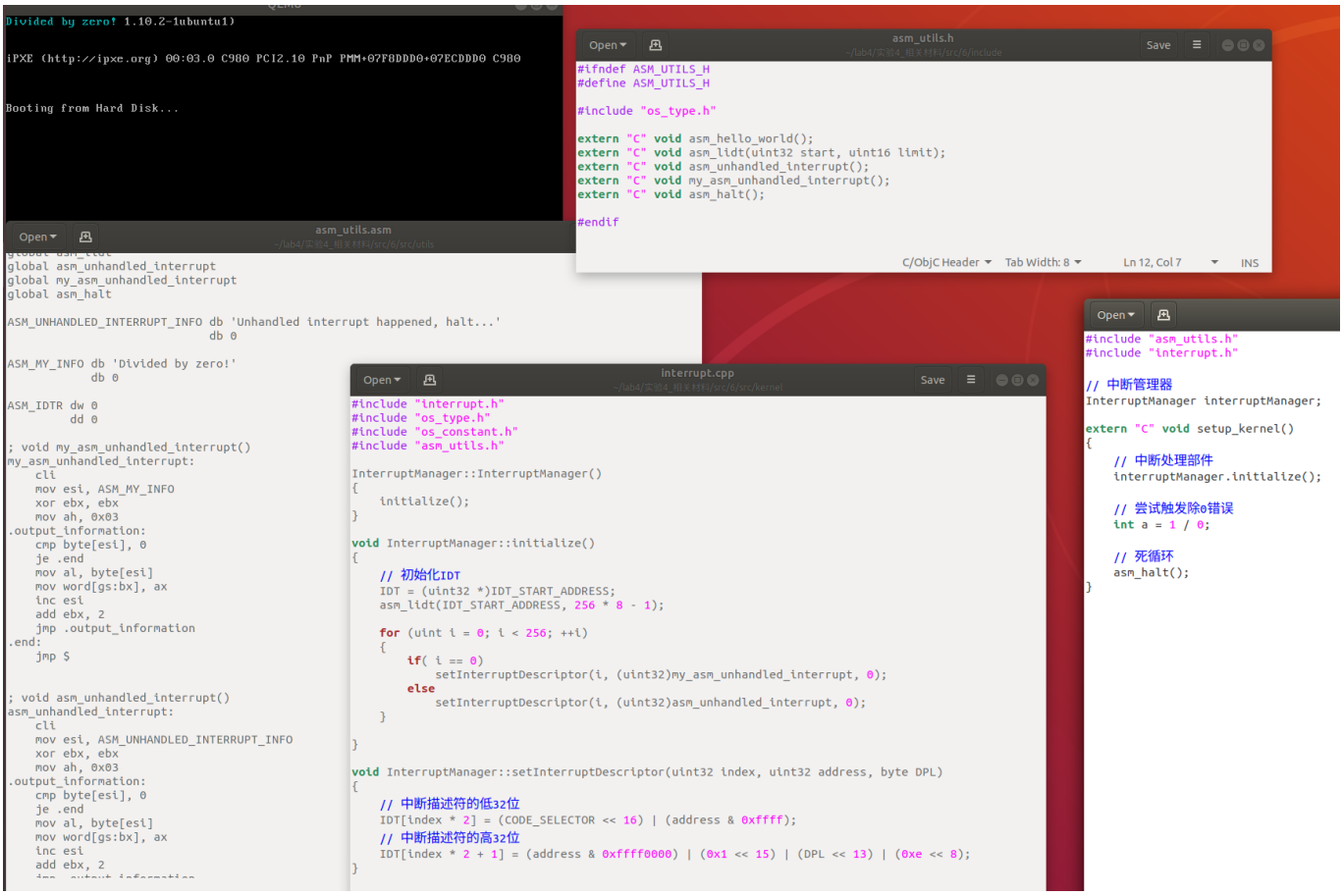
> - 初步了解C/C++项目的结构，以及各个文件夹的含义。按顺序编写makefile文件
> - mbr -> bootloader -> 进入kernel



在example2 代码的基础上，更改输出内容为姓名学号即可

assignment 3

> - 分析代码组织逻辑
> - 编写自定义中断处理函数并触发之



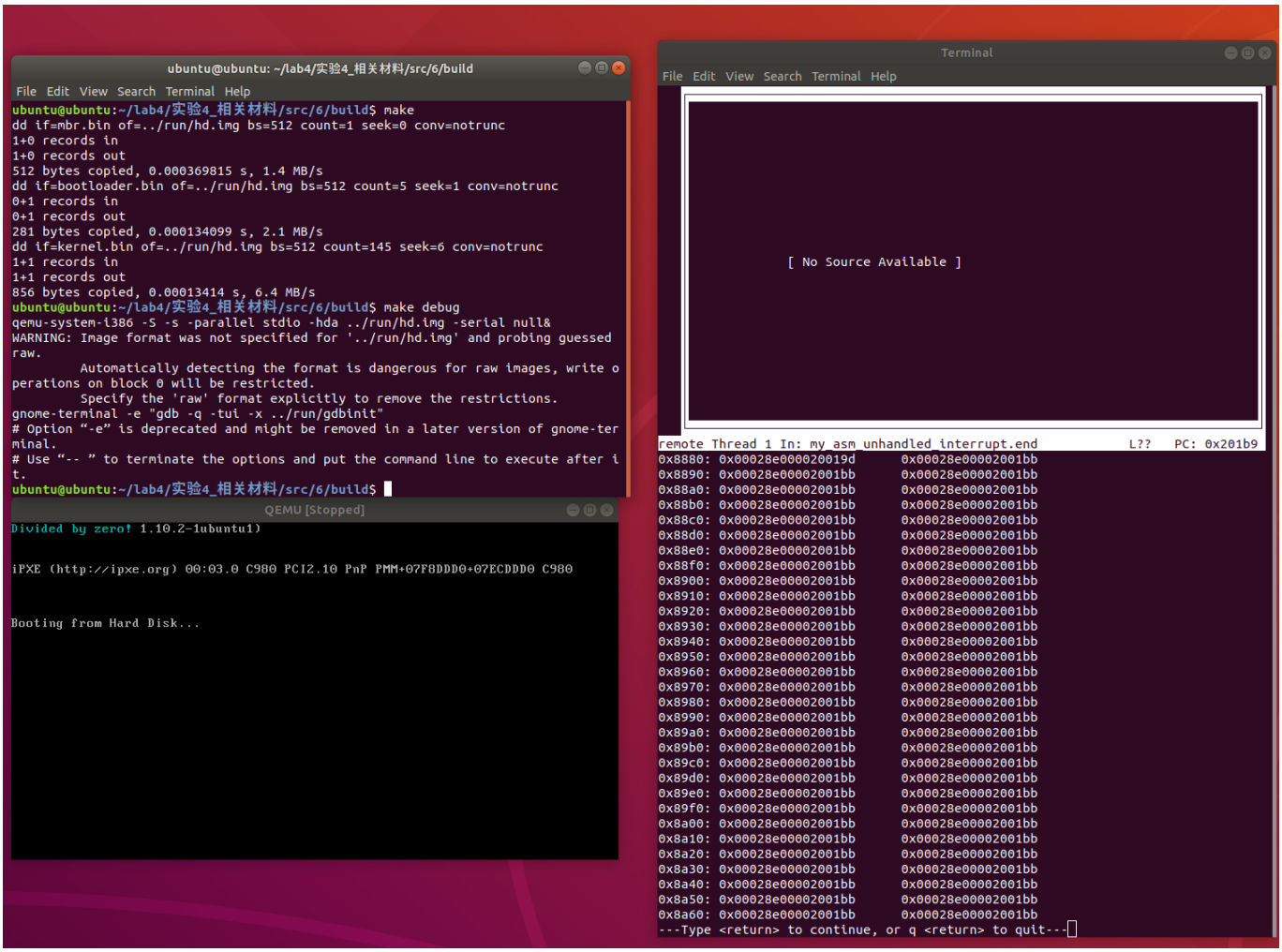
asm_utils.asm：自定义输出字符串`Divided by zero!`并将其输出

interrupt.cpp：初始化IDT时，仅对除零错误对应的中断处理函数进行更改，其余的还是按照`Unhandled interrupt happened, halt...`输出

asm_utils.h：添加语句`extern "C" void my_asm_unhandled_interrupt();`

> 关于gdb debug

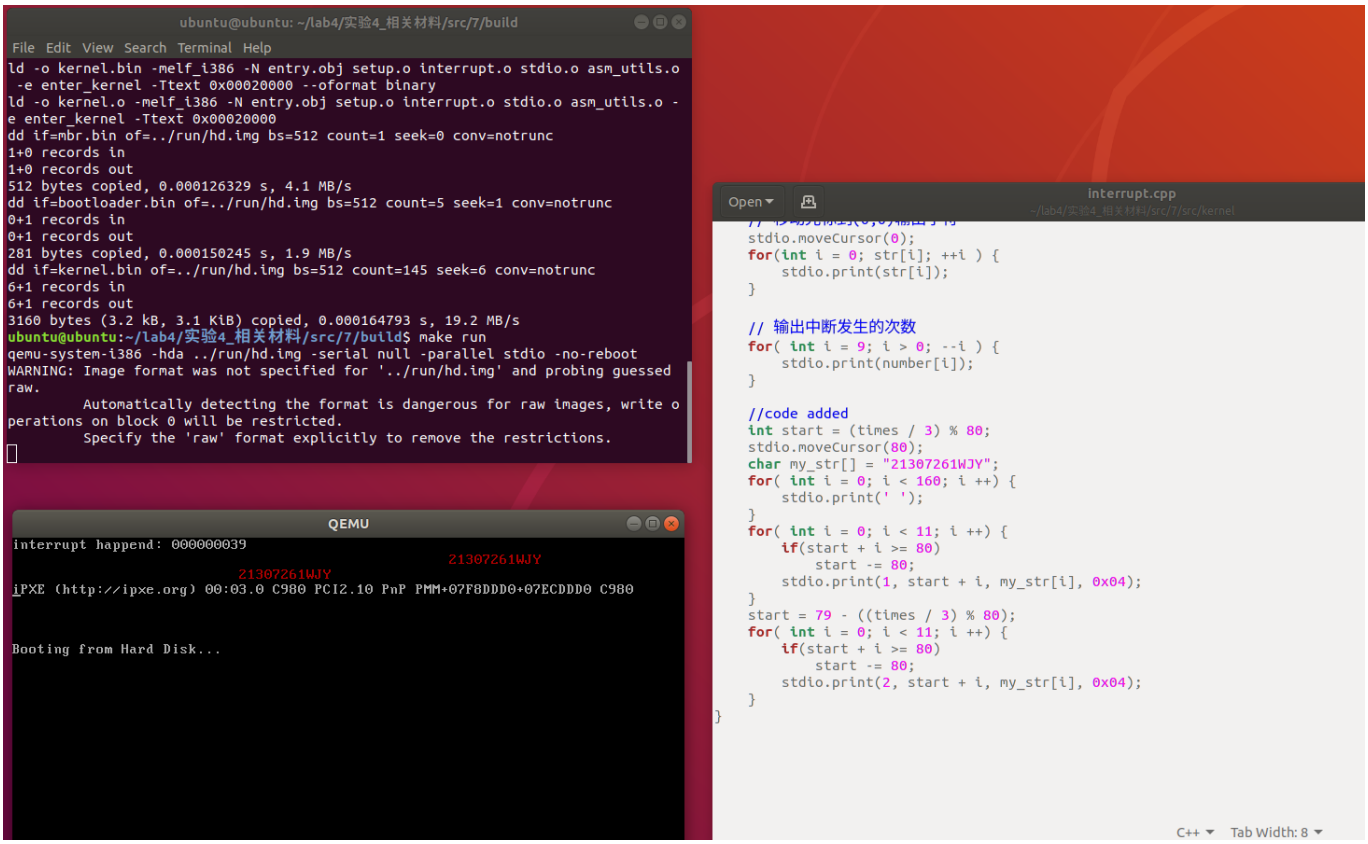
可以看到放入的64位中断描述符



assignment 4

> 实现简易的时钟中断，代码直接添加在`c_time_interrupt_handler`中

功能：在屏幕的第二行实现跑马灯，在第三行实现反向跑马灯



方式：借助times记录每次中断的开始打印位置。

因为每个中断打印一次，有点看的头晕眼花，所以采取了times / 3的形式将速度放慢了三倍

三、关键代码

（此处展示实验过程中有所修改的代码文件）

assignment1

asm_utils.asm

```
[bits 32]
global function_from_asm
global function_from_me_asm
extern function_from_C
extern function_from_CPP
extern function_from_me_C
extern function_from_me_CPP

function_from_asm:
    call function_from_C
    call function_from_CPP
    ret

; 添加的函数
function_from_me_asm:
    call function_from_me_C
```

```
    call function_from_me_CPP
    ret
```

c_func.c

```
#include <stdio.h>

void function_from_C() {
    printf("This is a function from C.\n");
}
// 添加的函数
void function_from_me_C() {
    printf("WJY\n");
}
```

cpp_func.cpp

```
#include <iostream>


extern "C" void function_from_CPP() {
    std::cout << "This is a function from C++." << std::endl; 添加的函数;
}
// 添加的函数
extern "C" void function_from_me_CPP() {
    std::cout << 21307261 << std::endl;
}
```

main.cpp

```
#include <iostream>

extern "C" void function_from_asm();
extern "C" void function_from_me_asm();

int main() {
    std::cout << "Call function from assembly." << std::endl;
    function_from_asm();
    function_from_me_asm();
    std::cout << "Done." << std::endl;
}
```

assignment 2

asm_utils.asm

```
[bits 32]

global asm_hello_world

asm_hello_world:
    push eax
    xor eax, eax

    mov ah, 0x03 ;青色
    mov al, '2'
    mov [gs:2 * 0], ax

    mov al, '1'
    mov [gs:2 * 1], ax

    mov al, '3'
    mov [gs:2 * 2], ax

    mov al, '0'
    mov [gs:2 * 3], ax

    mov al, '7'
    mov [gs:2 * 4], ax

    mov al, '2'
    mov [gs:2 * 5], ax

    mov al, '6'
    mov [gs:2 * 6], ax

    mov al, '1'
    mov [gs:2 * 7], ax

    mov al, 'W'
    mov [gs:2 * 8], ax

    mov al, 'J'
    mov [gs:2 * 9], ax

    mov al, 'Y'
    mov [gs:2 * 10], ax

    pop eax
    ret
```

assignment 3

asm_utils.asm

```
[bits 32]
```

```
global asm_hello_world
global asm_lidt
global asm_unhandled_interrupt
global my_asm_unhandled_interrupt
global asm_halt

ASM_UNHANDLED_INTERRUPT_INFO db 'Unhandled interrupt happened, halt...'
                             db 0

ASM_MY_INFO db 'Divided by zero!'
        db 0

ASM_IDTR dw 0
         dd 0

; 添加的函数
; void my_asm_unhandled_interrupt()
my_asm_unhandled_interrupt:
    cli
    mov esi, ASM_MY_INFO
    xor ebx, ebx
    mov ah, 0x03
.output_information:
    cmp byte[esi], 0
    je .end
    mov al, byte[esi]
    mov word[gs:bx], ax
    inc esi
    add ebx, 2
    jmp .output_information
.end:
    jmp $

; void asm_unhandled_interrupt()
asm_unhandled_interrupt:
    cli
    mov esi, ASM_UNHANDLED_INTERRUPT_INFO
    xor ebx, ebx
    mov ah, 0x03
.output_information:
    cmp byte[esi], 0
    je .end
    mov al, byte[esi]
    mov word[gs:bx], ax
    inc esi
    add ebx, 2
    jmp .output_information
.end:
    jmp $

; void asm_lidt(uint32 start, uint16 limit)
asm_lidt:
    push ebp
    mov ebp, esp
```

```
    push eax

    mov eax, [ebp + 4 * 3]
    mov [ASM_IDTR], ax
    mov eax, [ebp + 4 * 2]
    mov [ASM_IDTR + 2], eax
    lidt [ASM_IDTR]

    pop eax
    pop ebp
    ret

asm_hello_world:
    push eax
    xor eax, eax

    mov ah, 0x03 ;青色
    mov al, 'H'
    mov [gs:2 * 0], ax

    mov al, 'e'
    mov [gs:2 * 1], ax

    mov al, 'l'
    mov [gs:2 * 2], ax

    mov al, 'l'
    mov [gs:2 * 3], ax

    mov al, 'o'
    mov [gs:2 * 4], ax

    mov al, ' '
    mov [gs:2 * 5], ax

    mov al, 'W'
    mov [gs:2 * 6], ax

    mov al, 'o'
    mov [gs:2 * 7], ax

    mov al, 'r'
    mov [gs:2 * 8], ax

    mov al, 'l'
    mov [gs:2 * 9], ax

    mov al, 'd'
    mov [gs:2 * 10], ax

    pop eax
    ret
```

```
asm_halt:
    jmp $
```

setup.cpp

```
#include "asm_utils.h"
#include "interrupt.h"

// 中断管理器
InterruptManager interruptManager;

extern "C" void setup_kernel()
{
    // 中断处理部件
    interruptManager.initialize();

    // 尝试触发除0错误
    int a = 1 / 0;

    // 死循环
    asm_halt();
}
```

interrupt.cpp

```
#include "interrupt.h"
#include "os_type.h"
#include "os_constant.h"
#include "asm_utils.h"

InterruptManager::InterruptManager()
{
    initialize();
}

void InterruptManager::initialize()
{
    // 初始化IDT
    IDT = (uint32 *)IDT_START_ADDRESS;
    asm_lidt(IDT_START_ADDRESS, 256 * 8 - 1);

    for (uint i = 0; i < 256; ++i)
    {
        if( i == 0)
        setInterruptDescriptor(i, (uint32)my_asm_unhandled_interrupt, 0);
    else
        setInterruptDescriptor(i, (uint32)asm_unhandled_interrupt, 0);
    }
```

```
}

void InterruptManager::setInterruptDescriptor(uint32 index, uint32 address, byte
DPL)
{
    // 中断描述符的低32位
    IDT[index * 2] = (CODE_SELECTOR << 16) | (address & 0xffff);
    // 中断描述符的高32位
    IDT[index * 2 + 1] = (address & 0xffff0000) | (0x1 << 15) | (DPL << 13) | (0xe
<< 8);
}
```

assignment 4

部分interrupt.cpp

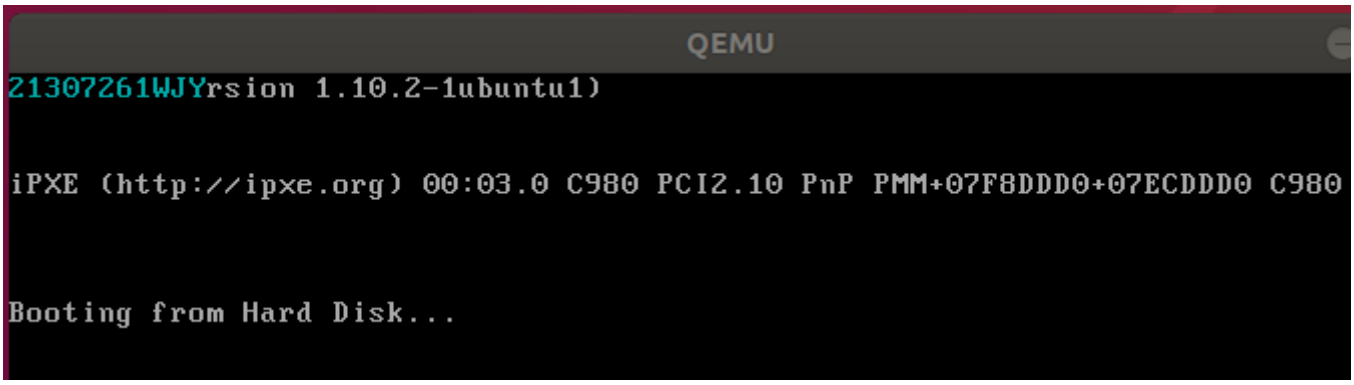
```
// 中断处理函数
extern "C" void c_time_interrupt_handler()
{
    // 清空屏幕
    for (int i = 0; i < 80; ++i)
    {
        stdio.print(0, i, ' ', 0x07);
    }

    // 输出中断发生的次数
    ++times;
    char str[] = "interrupt happend: ";
    char number[10];
    int temp = times;

    // 将数字转换为字符串表示
    for(int i = 0; i < 10; ++i ) {
        if(temp) {
            number[i] = temp % 10 + '0';
        } else {
            number[i] = '0';
        }
        temp /= 10;
    }

    // 移动光标到(0,0)输出字符
    stdio.moveCursor(0);
    for(int i = 0; str[i]; ++i ) {
        stdio.print(str[i]);
    }

    // 输出中断发生的次数
    for( int i = 9; i > 0; --i ) {
        stdio.print(number[i]);
    }
```

```
        //code added
        int start = (times / 3) % 80;
        stdio.moveCursor(80);
        char my_str[] = "21307261WJY";
        for( int i = 0; i < 160; i ++) {
        stdio.print(' ');
        }
        for( int i = 0; i < 11; i ++) {
        if(start + i >= 80)
            start -= 80;
        stdio.print(1, start + i, my_str[i], 0x04);
        }
        start = 79 - ((times / 3) % 80);
        for( int i = 0; i < 11; i ++) {
        if(start + i >= 80)
            start -= 80;
        stdio.print(2, start + i, my_str[i], 0x04);
        }
    }
```

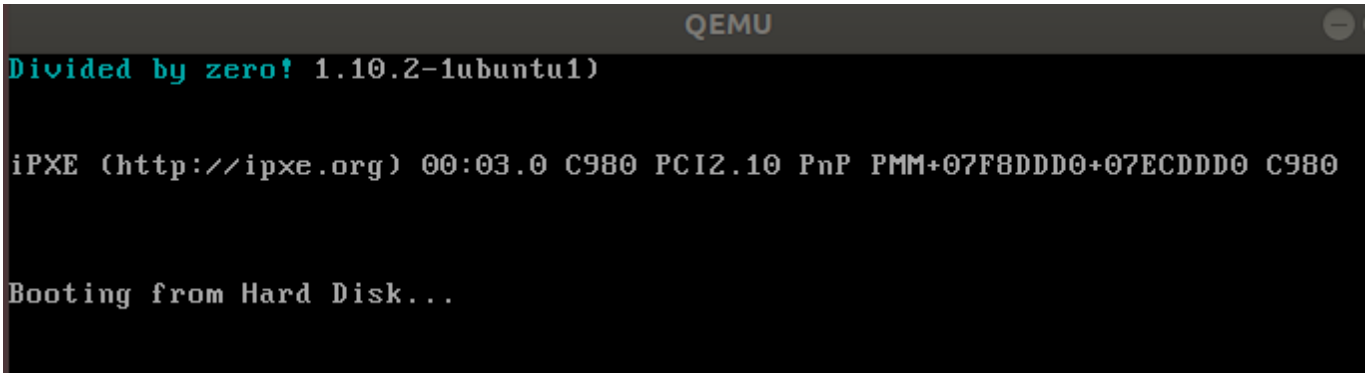
四、实验结果

assignment 1

assignment 2



assignment 3




```
remote Thread 1 In: my_asm_unhandled_interrupt.end          L??   PC: 0x201b9
0x8880: 0x00028e000020019d      0x00028e00002001bb
0x8890: 0x00028e00002001bb      0x00028e00002001bb
0x88a0: 0x00028e00002001bb      0x00028e00002001bb
0x88b0: 0x00028e00002001bb      0x00028e00002001bb
0x88c0: 0x00028e00002001bb      0x00028e00002001bb
0x88d0: 0x00028e00002001bb      0x00028e00002001bb
0x88e0: 0x00028e00002001bb      0x00028e00002001bb
0x88f0: 0x00028e00002001bb      0x00028e00002001bb
0x8900: 0x00028e00002001bb      0x00028e00002001bb
0x8910: 0x00028e00002001bb      0x00028e00002001bb
0x8920: 0x00028e00002001bb      0x00028e00002001bb
0x8930: 0x00028e00002001bb      0x00028e00002001bb
0x8940: 0x00028e00002001bb      0x00028e00002001bb
0x8950: 0x00028e00002001bb      0x00028e00002001bb
0x8960: 0x00028e00002001bb      0x00028e00002001bb
0x8970: 0x00028e00002001bb      0x00028e00002001bb
0x8980: 0x00028e00002001bb      0x00028e00002001bb
0x8990: 0x00028e00002001bb      0x00028e00002001bb
0x89a0: 0x00028e00002001bb      0x00028e00002001bb
0x89b0: 0x00028e00002001bb      0x00028e00002001bb
0x89c0: 0x00028e00002001bb      0x00028e00002001bb
0x89d0: 0x00028e00002001bb      0x00028e00002001bb
0x89e0: 0x00028e00002001bb      0x00028e00002001bb
0x89f0: 0x00028e00002001bb      0x00028e00002001bb
0x8a00: 0x00028e00002001bb      0x00028e00002001bb
0x8a10: 0x00028e00002001bb      0x00028e00002001bb
0x8a20: 0x00028e00002001bb      0x00028e00002001bb
0x8a30: 0x00028e00002001bb      0x00028e00002001bb
0x8a40: 0x00028e00002001bb      0x00028e00002001bb
0x8a50: 0x00028e00002001bb      0x00028e00002001bb
0x8a60: 0x00028e00002001bb      0x00028e00002001bb
---Type <return> to continue, or q <return> to quit---
```

可以看到，0号中断描述符低16位是`my_asm_unhandled_interrupt`的偏移地址；而往后的描述符低16位都是`asm_unhandled_interrupt`的偏移地址。这与`interrupt.cpp`里面的代码相符合

assignment 4

五、总结

这次的实验，让我深刻的了解了C代码到C程序的过程，对期间的四个阶段也有了新的认识。得益于这次的教程，让我得以从前三个代码文件中，一步一个脚印地去分析每条命令的作用。

教程由简入深，我的学习过程也是循序渐进。后面的几个用例均是以项目的形式来组织的，一开始我有点感到凌乱，文件的数量太多，复杂的makefile文件也使得我有点无从下手。后来，观看了一些makefile的教程之后，才逐渐理清了文件之间的大致逻辑关系。

例三例四则让我了解到了保护模式下的中断，从初始化到调用，并简单编写了一个中断程序，懂得了时钟中断的一些基础应用。